



To keep your account secure, move your 2FA to HTB Account by June 27th
The 2FA of Academy, will be disabled soon, so be sure you have 2FA enabled on SSO side.

[Enable now](#)

Finding & Filtering Content

Being able to search for, find, and filter content for what we are looking for is an absolute requirement for any user who utilizes the CLI (regardless of what shell or OS). Nevertheless, how do we do this in PowerShell? To answer this question, this section will dive into specifics of how PowerShell utilizes **Objects**, how we can **filter** based on **Properties** and **content**, and describe components like the PowerShell **Pipeline** further.

Explanation of PowerShell Output (Objects Explained)

With PowerShell, not everything is generic text strings like in Bash or cmd. In PowerShell, everything is an **Object**. However, what is an object? Let us examine this concept further:

What is an Object? An **object** is an **individual** instance of a **class** within PowerShell. Let us use the example of a computer as our object. The total of everything (parts, time, design, software, etc.) makes a computer a computer.

What is a Class? A class is the **schema** or 'unique representation of a thing (object) and how the sum of its **properties** define it. The **blueprint** used to lay out how that computer should be assembled and what everything within it can be considered a Class.

What are Properties? Properties are simply the **data** associated with an object in PowerShell. For our example of a computer, the individual **parts** that we assemble to make the computer are its properties. Each part serves a purpose and has a unique use within the object.

What are Methods? Simply put, methods are all the functions our object has. Our computer allows us to process data, surf the internet, learn new skills, etc. All of these are the methods for our object.

Now, we defined these terms so that we understand all the different properties we will be looking at later and what methods of interaction we have with objects. By understanding how PowerShell interprets objects and utilizes Classes, we can define our own object types. Moving on, we will look at how we can filter and find objects through the PowerShell CLI.

Finding and Filtering Objects

Let us look at this through a **user object** context. A user can do things like access files, run applications, and input/output data. But what is a user? What is it made up of?

Get an Object (User) and its Properties/Methods

Finding & Filtering Content		
PS C:\htb> Get-LocalUser administrator get-member		
TypeName: Microsoft.PowerShell.Commands.LocalUser		
Name	MemberType	Definition
----	-----	-----
Clone	Method	Microsoft.PowerShell.Commands.LocalUser Clone()
Equals	Method	bool Equals(System.Object obj)
GetHashCode	Method	int GetHashCode()
GetType	Method	type GetType()



To keep your account secure, move your 2FA to HTB Account by June 27th
The 2FA of Academy, will be disabled soon, so be sure you have 2FA enabled on SSO side.

[Enable now](#)

```
LastLogon           Property System.Nullable[datetime] LastLogon {get;set;}
Name                Property string Name {get;set;}
ObjectClass          Property string ObjectClass {get;set;}
PasswordChangeableDate Property System.Nullable[datetime] PasswordChangeableDate {get;set;}
PasswordExpires      Property System.Nullable[datetime] PasswordExpires {get;set;}
PasswordLastSet      Property System.Nullable[datetime] PasswordLastSet {get;set;}
PasswordRequired     Property bool PasswordRequired {get;set;}
PrincipalSource       Property System.Nullable[Microsoft.PowerShell.Commands.PrincipalSource] PrincipalSource
SID                  Property System.Security.Principal.SecurityIdentifier SID {get;set;}
UserMayChangePassword Property bool UserMayChangePassword {get;set;}
```

Now that we can see all of a user's properties let us look at what those properties look like when output by PowerShell. The [Select-Object](#) cmdlet will help us achieve this. In this manner, we now understand what makes up a user object.

Property Output (All)

Finding & Filtering Content

```
PS C:\htb> Get-LocalUser administrator | Select-Object -Property *

AccountExpires      :
Description          : Built-in account for administering the computer/domain
Enabled             : False
FullName            :
PasswordChangeableDate :
PasswordExpires      :
UserMayChangePassword : True
PasswordRequired     : True
PasswordLastSet      :
LastLogon           : 1/20/2021 5:39:14 PM
Name                 : Administrator
SID                  : S-1-5-21-3916821513-3027319641-390562114-500
PrincipalSource       : Local
ObjectClass          : User
```

A user is a small object realistically, but it can be a lot to look at the output in this manner, especially from items like large [lists](#) or [tables](#). So what if we wanted to filter this content down or show it to us in a more precise manner? We could filter out the properties of an object we do not want to see by selecting the few we do. Let's look at our users and see which have set a password recently.

Filtering on Properties

Finding & Filtering Content

```
PS C:\htb> Get-LocalUser * | Select-Object -Property Name,PasswordLastSet

Name                PasswordLastSet
----                -
Administrator
DefaultAccount
Guest
MTanaka              1/27/2021 2:39:55 PM
WDAGUtilityAccount  1/18/2021 7:40:22 AM
```

We can also [sort](#) and [group](#) our objects on these properties.

Sorting and Grouping



To keep your account secure, move your 2FA to HTB Account by June 27th
The 2FA of Academy, will be disabled soon, so be sure you have 2FA enabled on SSO side.

[Enable now](#)

Count	Name	Group
4	False	{Administrator, DefaultAccount, Guest, WDAGUtilityAccount}
1	True	{MTanaka}

We utilized the `Sort-Object` and `Group-Object` cmdlets to find all users, `sort` them by `name`, and then `group` them together based on their `Enabled` property. From the output, we can see that several users are disabled and not in use for interactive logon. This is just a quick example of what can be done with PowerShell objects and the sheer amount of information stored within each object. As we delve deeper into PowerShell and dig around within the Windows OS, we will notice that the classes behind many objects are extensive and often shared. Keep these things in mind as we work with them more and more.

Why Do We Need to Filter our Results?

We are switching it up and using an example of `get-service` for this demonstration. Looking at basic users and information does not produce much in the way of results, but other objects contain an extraordinary amount of data. Below is an example of just a fragment from the output of `Get-Service`:

Too Much Output

Finding & Filtering Content

```
PS C:\htb> Get-Service | Select-Object -Property *
```

```
Name                : AarSvc_1ca8ea
RequiredServices     : {}
CanPauseAndContinue : False
CanShutdown          : False
CanStop              : False
DisplayName           : Agent Activation Runtime_1ca8ea
DependentServices    : {}
MachineName          : .
ServiceName           : AarSvc_1ca8ea
ServicesDependedOn   : {}
ServiceHandle        :
Status               : Stopped
ServiceType           : 224
StartType            : Manual
Site                 :
Container            :
```

```
Name                : AdobeARMservice
RequiredServices     : {}
CanPauseAndContinue : False
CanShutdown          : False
CanStop              : True
DisplayName           : Adobe Acrobat Update Service
DependentServices    : {}
MachineName          : .
ServiceName           : AdobeARMservice
ServicesDependedOn   : {}
ServiceHandle        :
Status               : Running
ServiceType           : Win32OwnProcess
StartType            : Automatic
Site                 :
Container            :
```

```
Name                : agent_ovpnconnect
RequiredServices     : {}
```



To keep your account secure, move your 2FA to HTB Account by June 27th
The 2FA of Academy, will be disabled soon, so be sure you have 2FA enabled on SSO side.

[Enable now](#)

```
dependentServices : {}
MachineName       : .
ServiceName       : agent_ovpnconnect
ServicesDependedOn : {}
ServiceHandle     : 
Status            : Running
ServiceType       : Win32OwnProcess
StartType         : Automatic
Site              : 
Container         : 

<SNIP>
```

This is way too much data to sift through, right? Let us break it down further and format this data as a list. We can use the command string `get-service | Select-Object -Property DisplayName,Name,Status | Sort-Object DisplayName | fl` to change our output like so:

Finding & Filtering Content

```
PS C:\htb> get-service | Select-Object -Property DisplayName,Name,Status | Sort-Object DisplayName | fl

<SNIP>
DisplayName : ActiveX Installer (AxInstSV)
Name        : AxInstSV
Status      : Stopped

DisplayName : Adobe Acrobat Update Service
Name        : AdobeARMservice
Status      : Running

DisplayName : Adobe Genuine Monitor Service
Name        : AGMSERVICE
Status      : Running
<SNIP>
```

This is still a ton of output, but it is a bit more readable. Here is where we start asking ourselves questions like do we need all of this output? Do we care about all of these objects or just a specific subset of them? What if we wanted to determine if a specific service was running, but we needed to figure out the specific Name? The [Where-Object](#) can evaluate objects passed to it and their specific property values to look for the information we require. Consider this [scenario](#):

Scenario: We have just landed an initial shell on a host via an unsecured protocol exposing the host to the world. Before we get any further in, we need to assess the host and determine if any defensive services or applications are running. First, we look for any instance of `Windows Defender` services running.

Using [Where-Object](#) ([where](#) as an alias) and the parameter matching with `-like` will allow us to determine if we are safe to continue by looking for anything with "Defender" in the property. In this instance, we check the `DisplayName` property of all objects retrieved by `Get-Service`.

Hunting for Windows Defender

Finding & Filtering Content

```
PS C:\htb> Get-Service | where DisplayName -like '*Defender*'

Status Name                DisplayName
-----
-----
```



To keep your account secure, move your 2FA to HTB Account by June 27th
The 2FA of Academy, will be disabled soon, so be sure you have 2FA enabled on SSO side.

[Enable now](#)

As we can see, our results returned **several services running**, including Defender Firewall, Advanced Threat Protection, and more. This is both good news and bad news for us. We cannot just dive in and start doing things because we are likely to be spotted by the defensive services, but it is good that we spotted them and can now regroup and make a plan for defensive evasion actions to be taken. Although a quick example scenario, this is something as pentesters that we will often run into, and we should be able to spot and identify when defensive measures are in place. This example brings up an interesting way to modify our searches, however. Evaluation values can be beneficial to our cause. Let us check them out more.

The Evaluation of Values

Where and many other cmdlets can **evaluate** objects and data based on the values those objects and their properties contain. The output above is an excellent example of this utilizing the **-like** Comparison operator. It will look for anything that matches the values expressed and can include wildcards such as *****. Below is a quick list (not all-encompassing) of other useful expressions we can utilize:

Comparison Operators

Expression	Description
Like	Like utilizes wildcard expressions to perform matching. For example, ' *Defender* ' would match anything with the word Defender somewhere in the value.
Contains	Contains will get the object if any item in the property value matches exactly as specified.
Equal to	Specifies an exact match (case sensitive) to the property value supplied.
Match	Is a regular expression match to the value supplied.
Not	specifies a match if the property is blank or does not exist. It will also match False .

Of course, there are many other **comparison operators** we can use like, greater than, less than, and negatives like NotEqual, but in this kind of searching they may not be as widely used. Now with a **-GTE** understanding of how these operators can help us more than before (see what I did there), let us get back to digging into Defender services. Now we will look for service objects with a **DisplayName** again, like **< something>Defender< something>**.

Defender Specifics

Finding & Filtering Content	
<pre>PS C:\htb> Get-Service where DisplayName -like '*Defender*' Select-Object -Property *</pre>	
Name	: mpssvc
RequiredServices	: {mpsdrv, bfe}
CanPauseAndContinue	: False
CanShutdown	: False
CanStop	: False
DisplayName	: Windows Defender Firewall
DependentServices	:
MachineName	: .
ServiceName	: mpssvc
ServicesDependedOn	: {mpsdrv, bfe}
ServiceHandle	:
Status	: Running
ServiceType	: Win32ShareProcess
StartType	: Automatic
Site	:
...	:



To keep your account secure, move your 2FA to HTB Account by June 27th
The 2FA of Academy, will be disabled soon, so be sure you have 2FA enabled on SSO side.

[Enable now](#)

```
CanShutdown      : False
CanStop           : False
DisplayName       : Windows Defender Advanced Threat Protection Service
<SNIP>
```

Our results above now filter out every service associated with **Windows Defender** and displays the complete properties list of each match. Now we can look at the services, determine if they are running, and even if we can, at our current permission level, affect the status of those services (turn them off, disable them, etc.). During many of the commands we have issued in the last few sections, we have used the `|` symbol to concatenate multiple commands we would usually issue separately. Below we will discuss what this is and how it works for us.

What is the PowerShell Pipeline? (|)

In its simplest form, the **Pipeline** in PowerShell provides the end user a way to chain commands together. This chain is called a Pipeline and is also referred to as a pipe or piping commands together. With PowerShell handling objects the way it does, we can issue a command and then pipe (`|`) the resultant object output to another command for action. The Pipeline will interpret and execute the commands one at a time from left to right. We have done this in a few examples in the previous sections, so we are diving deeper into it here. As an example using the Pipeline to string commands together can look like this:

Piping Commands

Finding & Filtering Content

```
PS C:\htb> Command-1 | Command-2 | Command-3

Output from the result of 1+2+3
```

OR

Finding & Filtering Content

```
PS C:\htb>
Command-1 |
  Command-2 |
    Command-3

Output result from Pipeline
```

OR

Finding & Filtering Content

```
PS C:\htb> Get-Process | Where-Object CPU | Where-Object Path
              | Get-Item |

Output result from Pipeline
```

Each way is a perfectly acceptable way to concatenate the commands together. PowerShell can interpret what you want based on the position of the (`|`) in the string. Let us see an example of using the pipeline to provide us with actionable data. Below we will issue the **Get-Process** cmdlet, **sort** the resultant data, and then measure how many **unique** processes we have running on our host.



To keep your account secure, move your 2FA to HTB Account by June 27th
The 2FA of Academy, will be disabled soon, so be sure you have 2FA enabled on SSO side.

[Enable now](#)

Finding & Filtering Content

```
PS C:\htb> get-process | sort | unique | measure-object  
  
Count          : 113
```

As a result, the pipeline output the total count (**113**) of unique processes running at that time. Suppose we break the pipeline down at any particular point. In that case, we may see the process output sorted, filtered for unique instances (no duplicate names), or just a number output from the **Measure-Object** cmdlet. The task we performed was relatively simple. However, what if we could harness this for something more complex, like sorting new log entries, filtering for specific event log codes, or processing large amounts of data (a database and all its entries, for example) looking for specific strings? This is where Pipeline can increase our productivity and streamline the output we receive, making it a vital tool for any sysadmin or pentester.

Pipeline Chain Operators (**&&** and **||**)

Currently, Windows PowerShell 5.1 and older do not support Pipeline Chain Operators used in this fashion. If you see errors, you must install PowerShell 7 alongside Windows PowerShell. They are not the same thing.

You can find a great example of installing PowerShell 7 [here](#) so that you can use many of the new and updated features. PowerShell allows us to have conditional execution of pipelines with the use of **Chain operators**. These operators (**&&** and **||**) serve two main functions:

- **&&**: Sets a condition in which PowerShell will execute the next command inline **if** the current command **completes properly**.
- **||**: Sets a condition in which PowerShell will execute the following command inline **if** the current command **fails**.

These operators can be useful in helping us set conditions for scripts that execute if a goal or condition is met. For example:

Scenario: Let's say we write a command chain where we want to get the content within a file and then ping a host. We can set this to ping the host if the initial command succeeds with **&&** or to run only if the command fails **||**. Let's see both.

In this output, we can see that both commands were **successful** in execution because we get the output of the file **test.txt** printed to the console along with the results of our **ping** command.

Successful Pipeline

Finding & Filtering Content

```
PS C:\htb> Get-Content '.\test.txt' && ping 8.8.8.8  
pass or fail  
  
Pinging 8.8.8.8 with 32 bytes of data:  
Reply from 8.8.8.8: bytes=32 time=23ms TTL=118  
Reply from 8.8.8.8: bytes=32 time=28ms TTL=118  
Reply from 8.8.8.8: bytes=32 time=28ms TTL=118  
Reply from 8.8.8.8: bytes=32 time=21ms TTL=118  
  
Ping statistics for 8.8.8.8:  
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),  
Approximate round trip times in milli-seconds:  
    Minimum = 21ms, Maximum = 28ms, Average = 25ms
```

With this output we can see that our pipeline **closed** itself after the **first** command since it executed adequately printing the output



To keep your account secure, move your 2FA to HTB Account by June 27th
The 2FA of Academy, will be disabled soon, so be sure you have 2FA enabled on SSO side.

[Enable now](#)

Finding & Filtering Content

```
PS C:\htb> Get-Content '.\test.txt' || ping 8.8.8.8  
  
pass or fail
```

Here we can see that our pipeline executed **completely**. Our first command **failed** because the filename was typed wrong, and PowerShell sees this as the file we requested does not exist. Since the first command failed, our second command was executed.

Success in Failure

Finding & Filtering Content

```
PS C:\htb> Get-Content '.\testss.txt' || ping 8.8.8.8  
  
Get-Content: Cannot find path 'C:\Users\MTanaka\Desktop\testss.txt' because it does not exist.  
  
Pinging 8.8.8.8 with 32 bytes of data:  
Reply from 8.8.8.8: bytes=32 time=20ms TTL=118  
Reply from 8.8.8.8: bytes=32 time=37ms TTL=118  
Reply from 8.8.8.8: bytes=32 time=19ms TTL=118  
  
<SNIP>
```

The **pipeline** and **operators** that we used are beneficial to us from a time-saving perspective, as well as being able to quickly feed objects and data from one task to another. Issuing multiple commands in line is much more effective than manually issuing each command. What if we wanted to search for **strings** or **data** within the contents of files and directories? This is a common task many pentesters will perform while enumerating a host that they have gained access to. Searching with what is natively on the host is a great way to maintain our stealth and ensure we are not introducing new risks by bringing tools into the user environment.

Finding Data within Content

Some tools exist, like **Snaffler**, **Winpeas**, and the like, that can search for interesting files and strings, but what if we **cannot** bring a new tool onto the host? How can we hunt for sensitive info like credentials, keys, etc.? Combining cmdlets we have practiced in previous sections paired with new cmdlets like **Select-String** and **where** is an excellent way for us to root through a filesystem.

Select-String (**sls** as an alias) for those more familiar with using the Linux CLI, functions much in the same manner as **Grep** does or **findstr.exe** within the Windows Command-Prompt. It performs evaluations of input strings, file contents, and more based on regular expression (**regex**) pattern matching. When a match is found, **Select-String** will output the matching **line**, the **name** of the file, and the **line number** on which it was found by default. Overall it is a flexible and helpful cmdlet that should be in everyone's toolbox.

Below we will take our new cmdlet for a test drive as we look for information within some interesting files and directories that should be paid attention to when enumerating a host.

Find Interesting Files Within a Directory

When looking for interesting files, think about the most common file types we would use daily and start there. On a given day, we may write text files, a bit of Markdown, some Python, PowerShell, and many others. We want to look for those things when hunting through a host since it is where users and admins will interact most. We can start with **Get-ChildItem** and perform a recursive search through a folder. Let us test it out.



To keep your account secure, move your 2FA to HTB Account by June 27th
The 2FA of Academy, will be disabled soon, so be sure you have 2FA enabled on SSO side.

[Enable now](#)

Finding & Filtering Content

```
PS C:\htb> Get-ChildItem -Path C:\Users\MTanaka\ -File -Recurse

Directory: C:\Users\MTanaka\Desktop\notedump>NoteDump

Mode                LastWriteTime         Length Name
----                -
-a---            4/26/2022  1:47 PM          1092 demo notes.md
-a---            4/22/2022  2:20 PM          1074 noteDump.py
-a---            4/22/2022  2:55 PM        61440 plum.sqlite
-a---            4/22/2022  2:20 PM           375 README.md
<SNIP>
```

We will notice that it quickly returns way too much information. Every file in every folder in the path specified was output to our console. We need to trim this down a bit. Let us use the condition of looking at the **name** for specific **filetype extensions**. To do so, we will pipe the output of Get-ChildItem through the **where** cmdlet to filter down our output. Let's test first by searching for the ***.txt** filetype extension.

Narrowing Our Search

Finding & Filtering Content

```
PS C:\htb> Get-Childitem -Path C:\Users\MTanaka\ -File -Recurse -ErrorAction SilentlyContinue | where {($_.Name -like '*.txt')}

Directory: C:\Users\MTanaka\Desktop

Mode                LastWriteTime         Length Name
----                -
-a---            10/11/2022  3:32 PM          183 demo-notes.txt
-a---             4/4/2022  9:37 AM          188 q2-to-do.txt
-a---            10/12/2022  11:26 AM           14 test.txt
-a---             1/4/2022  11:23 PM          310 Untitled-1.txt

Directory: C:\Users\MTanaka\Desktop\win-stuff

Mode                LastWriteTime         Length Name
----                -
-a---            5/19/2021  10:12 PM        7831 wmic.txt

Directory: C:\Users\MTanaka\Desktop\Workshop\

Mode                LastWriteTime         Length Name
----                -
-----            1/7/2022  4:39 PM          945 info.txt
```

This worked much more efficiently. We only returned the files that matched the file type **txt** because of our filter's **\$_.Name** attribute. Now that we know it works, we can add the rest of the file types we will look for using an **-or** statement within the where filter.

Using **or** To Expand our Treasure Hunt

Finding & Filtering Content

```
PS C:\htb> Get-Childitem -Path C:\Users\MTanaka\ -File -Recurse -ErrorAction SilentlyContinue | where {($_.Name -like '*.txt' -or $_.Name -like '*.py' -or $_.Name -like '*.sqlite' -or $_.Name -like '*.md')}

Directory: C:\Users\MTanaka\Desktop

Mode                LastWriteTime         Length Name
----                -
-----            1/7/2022  4:39 PM          945 info.txt
```



To keep your account secure, move your 2FA to HTB Account by June 27th
The 2FA of Academy, will be disabled soon, so be sure you have 2FA enabled on SSO side.

[Enable now](#)

```
-a---          2/14/2022  3:40 PM          3824 remote-connect.ps1
-a---          10/11/2022  8:22 PM          874 treats.ps1
-a---          1/4/2022  11:23 PM          310 Untitled-1.txt
```

Directory: C:\Users\MTanaka\Desktop\notedump\NoteDump

Mode	LastWriteTime	Length	Name
-a---	4/26/2022 1:47 PM	1092	demo.md
-a---	4/22/2022 2:20 PM	1074	noteDump.py
-a---	4/22/2022 2:20 PM	375	README.md

Our string worked, and we are now retrieving **multiple filetypes** from Get-ChildItem! Now that we have our list of interesting files, we could turn around and **pipe** those objects into another cmdlet (**Select-String**) that searches through their content for interesting strings and keywords or phrases. Let us see this in action.

Basic Search Query

Finding & Filtering Content

```
PS C:\htb> Get-ChildItem -Path C:\Users\MTanaka\ -Filter "*.txt" -Recurse -File | sls "Password","credential",
CFP-Notes.txt:99:Lazzaro, N. (2004). Why we play games: Four keys to more emotion without story. Retrieved from
notes.txt:3:- Password: F@ll2022!
wmic.txt:67: wmic netlogin get name,badpasswordcount
wmic.txt:69:Are the screensavers password protected? What is the timeout? good use: see that all systems are
complying with policy evil use: find systems to walk up and use (assuming physical access is an option)
```

Keep in mind, Select-string is **not** case sensitive by default. If we wish for it to be, we can feed it the **-CaseSensitive** modifier. Now we will combine our original file search with our content filter.

Combining the Searches

Finding & Filtering Content

```
PS C:\htb> Get-Childitem -Path C:\Users\MTanaka\ -File -Recurse -ErrorAction SilentlyContinue | where {($_. Na
New-PC-Setup.md:56: - getting your vpn key
CFP-Notes.txt:99:Lazzaro, N. (2004). Why we play games: Four keys to more emotion without story. Retrieved from
notes.txt:3:- Password: F@ll2022!
wmic.txt:54: wmic computersystem get username
wmic.txt:67: wmic netlogin get name,badpasswordcount
wmic.txt:69:Are the screensavers password protected? What is the timeout? good use: see that all systems are
complying with policy evil use: find systems to walk up and use (assuming physical access is an option)
wmic.txt:83: wmic netuse get Name,username,connectiontype,localname
```

Our commands in the pipeline are getting longer, but we can easily clean up our view to make it readable. Looking at our results, though, it was a much smoother process to feed our file list results into our keyword search. Notice that there are a few **new** additions in our command string. We added a line to have the command continue if an error occurs (**-ErrorAction SilentlyContinue**). This helps us to ensure that our entire pipeline stays intact when it happens along a file or directory it cannot read. Finding and filtering content can be an interesting puzzle in and of itself. Determining what words and strings will produce the best results is an ever-evolving task and will often vary based on the customer.

Helpful Directories to Check



To keep your account secure, move your 2FA to HTB Account by June 27th
The 2FA of Academy, will be disabled soon, so be sure you have 2FA enabled on SSO side.

[Enable now](#)

- Looking in a Users `\AppData\` folder is a great place to start. Many applications store `configuration files`, `temp saves` of documents, and more.
- A Users home folder `C:\Users\User\` is a common storage place; things like VPN keys, SSH keys, and more are stored. Typically in `hidden` folders. (`Get-ChildItem -Hidden`)
- The Console History files kept by the host are an endless well of information, especially if you land on an administrator's host. You can check two different points:
 - `C:\Users\<USERNAME>\AppData\Roaming\Microsoft\Windows\Powershell\PSReadline\ConsoleHost_history.txt`
 - `Get-Content (Get-PSReadlineOption).HistorySavePath`
- Checking a user's clipboard may also yield useful information. You can do so with `Get-Clipboard`
- Looking at Scheduled tasks can be helpful as well.

These are just a few interesting places to check. Use it as a starting point to build and maintain your own checklist as your skill and experiences grow.

We are growing our CLI Kung Fu quickly, and it's time to move on to the next challenge. As you progress, please try the examples shown on your own to get a feel for what can be done and how you can modify them. We are jumping into working with Services and processes for our next lesson.

VPN Servers

 **Warning:** Each time you "Switch", your connection keys are regenerated and you must re-download your VPN connection file.

All VM instances associated with the old VPN Server will be terminated when switching to a new VPN server.

Existing PwnBox instances will automatically switch to the new VPN server.

US Academy 1

High Load



PROTOCOL

 UDP 1337  TCP 443

DOWNLOAD VPN CONNECTION FILE



Connect to Pwnbox

Your own web-based Parrot Linux instance to play our labs.

Pwnbox Location

AU

45ms



Terminate Pwnbox to switch location



To keep your account secure, move your 2FA to HTB Account by June 27th
The 2FA of Academy, will be disabled soon, so be sure you have 2FA enabled on SSO side.

Enable now

Start Instance

∞ / 1 spawns left

Waiting to start...



Questions

Answer the question(s) below to complete this Section and earn cubes!

Target(s): [Click here to spawn the target system!](#)



Cheat Sheet



Download VPN
Connection File

+ 0

What defines the functions our objects have?

Submit your answer here...

+10 Streak pts



Submit



Hint



Show Solution

+ 0

What Cmdlet can show us the properties and methods of an object?

get-member



Submit



Hint



Show Solution

+ 0

If we wanted to look through a directory and all sub-directories for something, what modifier would we use with the Get-ChildItem Cmdlet?

-recurse



Submit



Hint



Show Solution

← Previous

Next →



Cheat Sheet



Resources



To keep your account secure, move your 2FA to HTB Account by June 27th
The 2FA of Academy, will be disabled soon, so be sure you have 2FA enabled on SSO side.

[Enable now](#)

Introduction



CMD

Command Prompt Basics



Getting Help



System Navigation



Working with Directories and Files



Gathering System Information



Finding Files and Directories



Environment Variables



Managing Services



Working With Scheduled Tasks



PowerShell

CMD Vs. PowerShell



All About Cmdlets and Modules



User and Group Management



Working with Files and Directories



Finding & Filtering Content

Working with Services

Working with the Registry

Working with the Windows Event Log

Networking Management from The CLI

Interacting with The Web

PowerShell Scripting and Automation

Finish Strong

Skills Assessment

Beyond This Module

My Workstation

OFFLINE

Start Instance

/ 1 spawns left



To keep your account secure, move your 2FA to HTB Account by June 27th
The 2FA of Academy, will be disabled soon, so be sure you have 2FA enabled on SSO side.

[Enable now](#)